

Metode numerice

Sinteză completă Laboratoare 1–9 + Cheatsheet

stefan jipa

Ianuarie 2026

Cuprins

1	Introducere	3
2	Laborator 1 – Introducere în Python	3
2.1	Concepte de bază	3
2.2	Variabile și conversii	3
2.3	Operatori principali	3
2.4	Structuri de control	3
2.5	Funcții și lambda	4
2.6	Liste, tuple, dicționare, stringuri	4
2.7	Funcții built-in utile	4
2.8	Excepții și fișiere	4
3	Laborator 2 – Erori numerice	5
3.1	Tipuri de erori	5
3.2	Reprezentarea floating-point și epsilon-ul mașinii	5
3.3	Erori absolute și relative	5
3.4	Propagarea erorilor	5
3.5	Pierderea de semnificație	6
3.6	Condiționare și stabilitate	6
4	Laborator 3 – Calcul matriceal (NumPy)	6
4.1	Crearea matricelor	6
4.2	Operații elementare	6
4.3	Funcții liniare importante	6
4.4	Sisteme liniare și pseudoinversă	7
4.5	Valori și vectori proprii	7
5	Laborator 4 – Ecuații neliniare	7
5.1	Metoda biseției	7
5.2	Metoda Newton–Raphson	7
5.3	Metoda secantei	8
5.4	Metoda punctului fix	8
6	Laborator 5 – Sisteme liniare. Descompunere LU	8
6.1	Sistemul $Ax = b$	8
6.2	Descompunerea LU	9
6.3	Pivotare	9
6.4	Algoritm schematic LU	9
6.5	SciPy LU	9

7	Laborator 6 – Metoda Gauss pentru sisteme liniare	9
7.1	Matrice extinsă și operații elementare	9
7.2	Eliminare înainte și retro-substituție	10
8	Laborator 7 – Derivate și Integrale numerice	10
8.1	Derivare numerică	10
8.2	Integrare numerică	10
8.2.1	Metoda trapezelor	11
8.2.2	Metoda Simpson (1/3)	11
9	Laborator 8 – Interpolare și Regresie	11
9.1	Interpolare polinomială Lagrange	11
9.2	Interpolare Newton cu diferențe divizate	12
9.3	Interpolare spline cubic	12
9.4	Regresie liniară și polinomială	12
10	Laborator 9 – PCA (Principal Component Analysis)	13
10.1	Scopul PCA	13
10.2	Pași principali PCA	13
10.3	Varianța explicată	13
11	MEGA-CHEAT-SHEET (Esențial pentru examen)	14

1 Introducere

Acest document reprezintă o sinteză structurată a celor nouă laboratoare de Metode Numerice realizate în Python (Anaconda + Jupyter Notebook), împreună cu un *Mega-Cheat-Sheet* final pentru recapitulare rapidă.

Fiecare laborator conține:

- concepte teoretice esențiale;
- formule importante;
- fragmente de cod Python relevante;
- observații și capcane frecvente;
- o secțiune de tip “cheat sheet” local.

2 Laborator 1 – Introducere în Python

2.1 Concepte de bază

Python este un limbaj:

- interpretat, dinamic;
- multi-paradigmă (procedural, orientat obiect, funcțional);
- cu indentare semnificativă (delimitează blocuri de cod);
- folosit extensiv în știința datelor, simulări, metode numerice, ML etc.

Tipuri de bază:

- `int`, `float`, `str`, `bool`;
- colecții: `list`, `tuple`, `dict`, `set`.

2.2 Variabile și conversii

```
x = 5
y = 2.5
s = "text"
ok = True

a = int("5")
b = float("3.14")
c = str(10)
```

2.3 Operatori principali

Aritmetici:

`+`, `-`, `*`, `/`, `//`, `%`, `**`.

Comparare:

`==`, `!=`, `<`, `>`, `≤`, `≥`.

Logici:

`and`, `or`, `not`.

2.4 Structuri de control

```
if conditie:
    ...
elif alta_conditie:
    ...
```

```

else:
    ...

while conditie:
    ...

for i in range(10):
    print(i)

```

2.5 Funcții și lambda

```

def nume_functie(param1, param2):
    return param1 + param2

f = lambda x: x**2

```

2.6 Liste, tupluri, dicționare, stringuri

Liste:

```

lista = [1, 2, 3]
lista.append(4)
lista.insert(0, 10)
lista.remove(2)
lista.pop()
lista.sort()
patrate = [x*x for x in range(10)]

```

Tupluri:

```

t = (1, 2, 3)
a, b, c = t

```

Dicționare:

```

d = {"nume": "Ion", "varsta": 20}
nume = d["nume"]
varsta = d.get("varsta")
for cheie, valoare in d.items():
    print(cheie, valoare)

```

Stringuri:

```

s = "  Salut lume  "
s_upper = s.upper()
s_lower = s.lower()
s_strip = s.strip()
cuvinte = s.split()
joined = "-".join(cuvinte)

```

2.7 Funcții built-in utile

len(), type(), range(), print(), input(), sum(), min(), max(), abs().

2.8 Excepții și fișiere

```
try:
    num = int(input("Numar: "))
except ValueError:
    print("Nu ai introdus un numar valid!")

with open("fisier.txt", "r") as f:
    continut = f.read()
```

Cheat Sheet Lab 1 (scurt)

- Tipuri: int, float, str, bool, list, tuple, dict.
- Structuri: if, for, while, def, try/except.
- Liste: append, remove, sort, slicing.
- Dicționare: d[key], d.items().
- Stringuri: upper, lower, strip, split.
- Importuri: import numpy as np, import matplotlib.pyplot as plt.

3 Laborator 2 – Erori numerice

3.1 Tipuri de erori

- **Erori de modelare:** simplificarea realității printr-un model matematic.
- **Erori de discretizare:** apar la derivare numerică, integrare numerică etc.
- **Erori de rotunjire/trunchiere:** datorate reprezentării finite în virgulă mobilă.

3.2 Reprezentarea floating-point și epsilon-ul mașinii

Numerele reale sunt reprezentate ca:

$$x = \pm m \cdot b^e,$$

unde m este mantisa, b baza, e exponentul.

Machine epsilon:

$$\varepsilon_m = \min\{\varepsilon > 0 \mid 1 + \varepsilon > 1\}.$$

```
import numpy as np
eps = np.finfo(float).eps
```

3.3 Erori absolute și relative

$$E_{\text{abs}} = |A_{\text{real}} - A_{\text{calc}}|, \quad E_{\text{rel}} = \frac{|A_{\text{real}} - A_{\text{calc}}|}{|A_{\text{real}}|}.$$

3.4 Propagarea erorilor

La adunare/scădere:

$$E \approx E_1 + E_2.$$

La înmulțire/împărțire (erori relative):

$$E_{\text{rel}} \approx E_{\text{rel},1} + E_{\text{rel},2}.$$

3.5 Pierderea de semnificație

Apare la scăderea numerelor apropiate:

$$f(x) = \frac{1 - \cos x}{x^2}, \quad x \approx 0,$$

unde devine preferabilă formula aproximativă:

$$1 - \cos x \approx \frac{x^2}{2}.$$

3.6 Condiționare și stabilitate

Condiționare: sensibilitatea problemei la perturbări ale datelor de intrare. Un coeficient tipic de condiționare:

$$\text{cond}(x) = \left| \frac{x f'(x)}{f(x)} \right|.$$

Stabilitate: modul în care algoritmul amplifică (sau nu) erorile; un algoritm instabil poate da rezultate proaste chiar pentru o problemă bine condiționată.

Cheat Sheet Lab 2 (scurt)

- $E_{\text{abs}} = |A - A'|$, $E_{\text{rel}} = E_{\text{abs}}/|A|$.
- `np.finfo(float).eps` pentru epsilon.
- Evită scăderea numerelor foarte apropiate.
- Condiționare bună $\Rightarrow$ soluții stabile la perturbări mici.

4 Laborator 3 – Calcul matriceal (NumPy)

4.1 Crearea matricelor

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
Z = np.zeros((m, n))
O = np.ones((m, n))
I = np.eye(n)
R = np.random.rand(m, n)
```

4.2 Operații elementare

```
C1 = A + B
C2 = A - B
C3 = A * B           # produs element-wise
C4 = A @ B           # produs matricial
AT = A.T             # transpusa
```

4.3 Funcții liniare importante

```
detA = np.linalg.det(A)
invA = np.linalg.inv(A)
rankA = np.linalg.matrix_rank(A)

normF = np.linalg.norm(A)           # Frobenius
norm1 = np.linalg.norm(A, 1)        # norma 1
normInf = np.linalg.norm(A, np.inf)
```

4.4 Sisteme liniare și pseudoinversă

```
x = np.linalg.solve(A, b)      # A trebuie sa fie patratice i inversabil
A_pinv = np.linalg.pinv(A)     # pseudoinvers (Moore-Penrose)
```

4.5 Valori și vectori proprii

```
vals, vecs = np.linalg.eig(A)
```

Pentru diagonalizare:

$$A = PDP^{-1}, \quad D = \text{diag}(\lambda_i).$$

Cheat Sheet Lab 3 (scurt)

- Produs: $A @ B$, transpusa: $A.T$.
- `np.linalg.det`, `np.linalg.inv`, `np.linalg.solve`.
- `np.linalg.eig` pentru valori/vectori proprii.
- `np.linalg.pinv` pentru pseudoinversă.

5 Laborator 4 – Ecuatii neliniare

Considerăm ecuația:

$$f(x) = 0.$$

5.1 Metoda biseției

Condiție: $f(a)f(b) < 0$.

$$c = \frac{a+b}{2}, \quad \begin{cases} b \leftarrow c & \text{dacă } f(a)f(c) < 0, \\ a \leftarrow c & \text{altfel.} \end{cases}$$

Criterii de oprire:

$$|f(c)| < \varepsilon \quad \text{sau} \quad |b-a| < \varepsilon.$$

```
def bisectie(f, a, b, eps):
    while (b - a) > eps:
        c = (a + b) / 2
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return c
```

5.2 Metoda Newton–Raphson

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

- Convergență rapidă (cvasi-quadratică) dacă x_0 este aproape de rădăcină.
- Necesită derivata $f'(x)$ și $f'(x) \neq 0$ în vecinătate.

```
def newton(f, df, x0, eps, max_iter=100):
    x = x0
    for _ in range(max_iter):
        if abs(f(x)) < eps:
            break
        x = x - f(x) / df(x)
    return x
```

5.3 Metoda secantei

Aproximare Newton fără derivată:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

```
def secanta(f, x0, x1, eps, max_iter=100):
    for _ in range(max_iter):
        if abs(x1 - x0) < eps:
            break
        x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
        x0, x1 = x1, x2
    return x1
```

5.4 Metoda punctului fix

Scriem:

$$x = g(x),$$

și iterăm:

$$x_{k+1} = g(x_k).$$

Condiție tipică de convergență:

$$|g'(x)| < 1 \quad \text{în zona rădăcinii.}$$

```
def punct_fix(g, x0, eps, max_iter=100):
    x = x0
    for _ in range(max_iter):
        x_next = g(x)
        if abs(x_next - x) < eps:
            break
        x = x_next
    return x
```

Cheat Sheet Lab 4 (scurt)

- Biseecție: sigură, dar lentă.
- Newton: rapidă, dar sensibilă la x_0 .
- Secantă: nu cere derivată, viteză bună.
- Punct fix: necesită $|g'| < 1$.

6 Laborator 5 – Sisteme liniare. Descompunere LU

6.1 Sistemul $Ax = b$

A este matrice pătratică, x vector necunoscut, b vector termeni liberi.

6.2 Descompunerea LU

$$A = LU,$$

unde L este triunghiulară inferioară (cu 1 pe diagonală), iar U triunghiulară superioară.

Rezolvare:

$$Ly = b, \quad Ux = y.$$

6.3 Pivotare

Pentru stabilitate numerică:

$$PA = LU,$$

unde P este matrice de permutare (pivotare parțială).

6.4 Algoritm schematic LU

```
import numpy as np

A = A.astype(float)
n = A.shape[0]
L = np.eye(n)
U = A.copy()

for k in range(n-1):
    # aici se poate introduce pivotare
    for i in range(k+1, n):
        L[i, k] = U[i, k] / U[k, k]
        U[i, :] = U[i, :] - L[i, k] * U[k, :]
```

6.5 SciPy LU

```
import scipy.linalg as la

P, L, U = la.lu(A)
```

Cheat Sheet Lab 5 (scurt)

- $A = LU$ sau $PA = LU$.
- Rezolvare: $Ly = b$, apoi $Ux = y$.
- Pivotare pentru stabilitate.

7 Laborator 6 – Metoda Gauss pentru sisteme liniare

7.1 Matrice extinsă și operații elementare

Scriem sistemul:

$$Ax = b$$

sub forma matricei extinse:

$$[A|b].$$

Operații elementare pe linii:

- schimb de linii: $L_i \leftrightarrow L_j$;
- înmulțire linie cu scalar nenul;
- adunare de multiplu al altei linii: $L_i \leftarrow L_i - mL_k$.

7.2 Eliminare înainte și retro-substituție

```
import numpy as np

def gauss(A, b):
    A = A.astype(float)
    b = b.astype(float)
    n = len(b)

    # eliminare
    for k in range(n-1):
        # pivotare optionala
        for i in range(k+1, n):
            factor = A[i, k] / A[k, k]
            A[i, k:] = A[i, k:] - factor * A[k, k:]
            b[i] = b[i] - factor * b[k]

    # retro-substituție
    x = np.zeros(n)
    x[-1] = b[-1] / A[-1, -1]
    for i in range(n-2, -1, -1):
        s = b[i] - np.dot(A[i, i+1:], x[i+1:])
        x[i] = s / A[i, i]
    return x
```

Cheat Sheet Lab 6 (scurt)

- Transformați $[A|b]$ în formă triunghiulară.
- Apoi aplicați retro-substituție.
- Situații speciale: rând de forma $[0 \ 0 \ \dots \ |c]$ cu $c \neq 0 \Rightarrow$ sistem incompatibil.

8 Laborator 7 – Derivate și Integrale numerice

8.1 Derivare numerică

Diferență înainte:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \quad O(h).$$

Diferență înapoi:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}, \quad O(h).$$

Diferență centrală:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \quad O(h^2).$$

```
def deriv_forward(f, x, h):
    return (f(x + h) - f(x)) / h

def deriv_backward(f, x, h):
    return (f(x) - f(x - h)) / h

def deriv_central(f, x, h):
    return (f(x + h) - f(x - h)) / (2*h)
```

8.2 Integrare numerică

Considerăm:

$$\int_a^b f(x) dx.$$

8.2.1 Metoda trapezelor

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right),$$

unde $h = \frac{b-a}{n}$.

```
import numpy as np

def trapez(f, a, b, n):
    x = np.linspace(a, b, n+1)
    h = (b - a) / n
    return h * (0.5*f(x[0]) + np.sum(f(x[1:-1])) + 0.5*f(x[-1]))
```

8.2.2 Metoda Simpson (1/3)

$$\int_a^b f(x) dx \approx \frac{h}{3} \left(f(a) + 4 \sum_{\text{indice impar}} f(x_i) + 2 \sum_{\text{indice par}} f(x_i) + f(b) \right),$$

cu n par.

```
def simpson(f, a, b, n):
    if n % 2 == 1:
        raise ValueError("n trebuie sa fie par.")
    x = np.linspace(a, b, n+1)
    h = (b - a) / n
    return (h/3) * (f(x[0])
                    + 4*np.sum(f(x[1:-1:2]))
                    + 2*np.sum(f(x[2:-2:2]))
                    + f(x[-1]))
```

Cheat Sheet Lab 7 (scurt)

- Derivată centrală: cea mai precisă.
- Integrale: Trapez = simplu, Simpson = precis ($O(h^4)$).
- h prea mare $\Rightarrow$ eroare de discretizare; h prea mic $\Rightarrow$ eroare de rotunjire.

9 Laborator 8 – Interpolare și Regresie

9.1 Interpolare polinomială Lagrange

Puncte:

$$(x_0, y_0), \dots, (x_n, y_n).$$

Polinomul de interpolare:

$$P_n(x) = \sum_{k=0}^n y_k L_k(x),$$

unde

$$L_k(x) = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}.$$

```
def lagrange(x, xi, yi):
    s = 0
    n = len(xi)
    for k in range(n):
        L = 1
        for j in range(n):
            if j != k:
                L *= (x - x_j) / (x_k - x_j)
```

```

        L *= (x - xi[j])/(xi[k] - xi[j])
    s += yi[k] * L
return s

```

9.2 Interpolare Newton cu diferențe divizate

Formă:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots$$

```

def coef_newton(x, y):
    n = len(x)
    a = y.copy()
    for k in range(1, n):
        a[k:n] = (a[k:n] - a[k-1])/(x[k:n] - x[k-1])
    return a

def newton_eval(a, x, xi):
    p = a[-1]
    for k in range(len(a)-2, -1, -1):
        p = p*(x - xi[k]) + a[k]
    return p

```

9.3 Interpolare spline cubic

```

from scipy.interpolate import CubicSpline

cs = CubicSpline(x, y)
y_new = cs(x_new)

```

Spline cubic oferă continuitate de ordin 0, 1, 2 și evită oscilațiile polinoamelor de grad mare.

9.4 Regresie liniară și polinomială

Regresie liniară: $y \approx ax + b$, obținută prin metoda celor mai mici pătrate.

Formulele clasice:

$$a = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad b = \frac{\sum y_i - a \sum x_i}{n}.$$

```

import numpy as np

def regresie liniara(x, y):
    n = len(x)
    a = (n*np.sum(x*y) - np.sum(x)*np.sum(y)) / \
        (n*np.sum(x**2) - (np.sum(x))**2)
    b = (np.sum(y) - a*np.sum(x)) / n
    return a, b

# varianta cu np.polyfit
coef = np.polyfit(x, y, 1) # regresie liniara

```

Regresie polinomială (grad k):

$$P_k(x) = a_0 + a_1x + \dots + a_kx^k.$$

```

p = np.polyfit(x, y, k)
y_est = np.polyval(p, x_new)

```

Cheat Sheet Lab 8 (scurt)

- Interpolare = trece exact prin puncte (Lagrange, Newton, spline).
- Regresie = ajustează o curbă care minimizează eroarea (least squares).
- `np.polyfit`, `np.polyval` pentru regresie.
- `CubicSpline` pentru interpolare stabilă.

10 Laborator 9 – PCA (Principal Component Analysis)

10.1 Scopul PCA

PCA este o tehnică de:

- reducere a dimensionalității;
- găsire a direcțiilor cu varianță maximă;
- proiecție a datelor într-un spațiu cu mai puține dimensiuni, cu pierdere minimă de informație.

10.2 Pași principali PCA

1. Standardizare: $X_{\text{std}} = \frac{X - \mu}{\sigma}$.
2. Calcul matrice de covarianță: $C = \frac{1}{n-1} X^T X$.
3. Calcul valori și vectori proprii: λ_i, v_i .
4. Sortare descrescătoare după λ_i .
5. Alegere primele k componente (vectori proprii).
6. Proiecție: $X_{\text{PCA}} = X_{\text{std}} W$, unde W conține primele k vectori proprii.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import numpy as np

scaler = StandardScaler()
X_std = scaler.fit_transform(X)

pca = PCA(n_components=k)
X_pca = pca.fit_transform(X_std)

components = pca.components_
expl_var = pca.explained_variance_ratio_
```

10.3 Varianța explicată

Varianța explicată de componenta i : $\frac{\lambda_i}{\sum_j \lambda_j}$.

În sklearn:

```
pca.explained_variance_ratio_
```

Cheat Sheet Lab 9 (scurt)

- Standardizezi datele.
- Calculezi matricea de covarianță și eigen-decompoziția.
- Alegi k componente cu varianță explicată mare.
- Proiectezi: $X_{\text{std}} W$ sau `pca.fit_transform`.

11 MEGA-CHEAT-SHEET (Esențial pentru examen)

1. Python & bază

- Tipuri: `int`, `float`, `str`, `bool`, `list`, `tuple`, `dict`.
- Control: `if`, `for`, `while`, `def`, `try/except`.
- Importuri standard: `numpy`, `matplotlib`, `scipy`, `sklearn`.

2. Erori numerice

$$E_{\text{abs}} = |A - A|, \quad E_{\text{rel}} = \frac{E_{\text{abs}}}{|A|}, \quad \varepsilon_m = \text{np.finfo(float).eps}.$$

Evitați:

- scăderea numerelor foarte apropiate (pierdere de semnificație);
- expresii instabile precum $\frac{1 - \cos x}{x^2}$ pentru x mic.

3. Calcul matriceal

- Produs: `A @ B`, transpusa: `A.T`.
- Determinant: `np.linalg.det(A)`.
- Inversă: `np.linalg.inv(A)` (evitat dacă se poate).
- Sisteme: `np.linalg.solve(A, b)` – preferat în loc de inversă.
- Eigen: `np.linalg.eig(A)`.

4. Ecuatii neliniare

- Biseție: sigură, necesită semne opuse $f(a)f(b) < 0$.
- Newton: $x_{k+1} = x_k - f(x_k)/f'(x_k)$, rapid dar sensibil la x_0 .
- Secantă: aproximare Newton fără derivată.
- Punct fix: $x = g(x)$, necesită $|g'(x)| < 1$.

5. Sisteme liniare (Gauss, LU)

- Gauss: transformă $[A|b]$ în formă triunghiulară, apoi retro-substituție.
- LU: $A = LU$ sau $PA = LU$, rezolvi $Ly = b$, apoi $Ux = y$.
- Pivotare pentru stabilitate numerică.

6. Derivate și Integrale

- Derivare:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \text{ (centrală).}$$

- Integrare trapez:

$$\frac{h}{2} (f(a) + 2 \sum f(x_i) + f(b)).$$

- Integrare Simpson (1/3):

$$\frac{h}{3} (f(a) + 4 \sum_{\text{impar}} f(x_i) + 2 \sum_{\text{par}} f(x_i) + f(b)).$$

7. Interpolare și Regresie

- Interpolare Lagrange/Newton: polinom care trece prin toate punctele.
- Spline cubic: interpolare stabilă, fără oscilații mari.
- Regresie liniară/polinomială:
 - `np.polyfit(x, y, k)` $\Rightarrow$ coeficienți;
 - `np.polyval(p, x_new)` $\Rightarrow$ evaluare.

8. PCA

- Standardizezi datele (mediană, deviație standard).
- Calculezi matricea de covarianță și decompoziția eigen.
- Sortezi componentele după varianța explicată.
- Alegi k componente principale și proiectezi datele.
- În sklearn: `PCA(n_components=k)`, `fit_transform`.